

[BUG] Error when taking `adjoint` of subroutines with `wires` arguments

<https://github.com/PennyLaneAI/catalyst/issues/334>

ROW 97

[BUG] Error when taking adjoint of subroutines with wires arguments #334

[New issue](#)[Closed](#)

#432



glassnotes opened on Oct 25, 2023

...

Issue description

- *Expected behavior:* A list of wires can be passed to a subroutine, and the subroutine executed both in its "regular" form as well as `adjoint` when `qjit` ting.
- *Actual behavior:* Errors are thrown in the `adjoint` call.
- *Reproduces how often:* (What percentage of the time does it reproduce?) Always
- *System information:* (post the output of `import pennylane as qml; qml.about()`)

I am on [this branch/commit](#) but was experiencing this previously on the main branch as well.

```
Name: PennyLane
Version: 0.32.0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/PennyLaneAI/pennylane
Author:
Author-email:
License: Apache License 2.0
Location: /home/olivia/Software/anaconda3/envs/catalyst/lib/python3.11/site-packages
Requires: appdirs, autograd, autoray, cachetools, networkx, numpy, pennylane-lightning, requests, rustworkx, s
Required-by: pennylane-catalyst, PennyLane-Lightning, PennyLane-qiskit
```

```
Platform info:      Linux-6.2.0-35-generic-x86_64-with-glibc2.35
Python version:     3.11.0
Numpy version:      1.23.5
Scipy version:      1.10.1
Installed devices:
- default.gaussian (PennyLane-0.32.0)
- default.mixed (PennyLane-0.32.0)
- default.qubit (PennyLane-0.32.0)
- default.qubit.autograd (PennyLane-0.32.0)
- default.qubit.jax (PennyLane-0.32.0)
- default.qubit.tf (PennyLane-0.32.0)
- default.qubit.torch (PennyLane-0.32.0)
- default.qutrit (PennyLane-0.32.0)
- null.qubit (PennyLane-0.32.0)
- qiskit.aer (PennyLane-qiskit-0.32.0)
- qiskit.basicaer (PennyLane-qiskit-0.32.0)
- qiskit.ibmq (PennyLane-qiskit-0.32.0)
- qiskit.ibmq.circuit_runner (PennyLane-qiskit-0.32.0)
- qiskit.ibmq.sampler (PennyLane-qiskit-0.32.0)
- qiskit.remote (PennyLane-qiskit-0.32.0)
- lightning.qubit (PennyLane-Lightning-0.32.0)
```

Source code and tracebacks

This version works:

```
import catalyst
import pennylane as qml
from catalyst import qjit, adjoint

dev = qml.device("lightning.qubit", wires=3)

def subroutine_no_wires():
    for wire in range(3):
        qml.PauliX(wire)

@qjit(autograph=True)
@qml.qnode(dev)
def test_function():
    adjoint(subroutine_no_wires)()
    return qml.probs()
```

However, if we instead do:

Assignees

No one assigned

Labels

[bug](#)

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

[\[Tests\] Check the PyTree-compatibility of Wires](#)
PennyLaneAI/catalyst

Notifications

[Customize](#)[Subscribe](#)

You're not receiving notifications from this thread.

Participants



```
def subroutine(wires):
    for wire in wires:
        qml.PauliX(wire)

@qjit(autograph=True)
@qml.qnode(dev)
def test_function():
    adjoint(subroutine)(dev.wires)
    return qml.probs()
```

we obtain the following traceback:

```
-----
KeyError                                Traceback (most recent call last)
File ~/Software/anaconda3/envs/catalyst/lib/python3.11/site-packages/jax/_src/api_util.py:581, in shaped_abstractify
    580 try:
--> 581     return _shaped_abstractify_handlers[type(x)](x)
    582 except KeyError:

KeyError: <class 'pennylane.wires.Wires'>

During handling of the above exception, another exception occurred:

TypeError                                Traceback (most recent call last)
Cell In[6], line 5
      2     for wire in wires:
      3         qml.PauliX(wire)
----> 5 @qjit(autograph=True)
      6 @qml.qnode(dev)
      7 def test_function():
      8     subroutine(dev.wires)
      9     adjoint(subroutine)(dev.wires)

File ~/Code/catalyst/frontend/catalyst/compilation_pipelines.py:948, in qjit.<locals>.wrap_fn(fn)
    947 def wrap_fn(fn):
--> 948     return QJIT(
    949         fn, CompileOptions(verbose, logfile, target, keep_intermediate, pipelines, autograph)
    950     )

File ~/Code/catalyst/frontend/catalyst/compilation_pipelines.py:508, in QJIT.__init__(self, fn, compile_options)
    506 if parameter_types is not None:
    507     self.user_typed = True
--> 508     self.mlir_module = self.get_mlir(*parameter_types)
    509     if self.compile_options.target == "binary":
    510         self.compiled_function = self.compile()

File ~/Code/catalyst/frontend/catalyst/compilation_pipelines.py:555, in QJIT.get_mlir(self, *args)
    550 self.c_sig = CompiledFunction.get_runtime_signature(*args)
    552 with Patcher(
    553     (qml.QNode, "__call__", QFunc.__call__),
    554 ):
--> 555     mlir_module, ctx, jaxpr, self.shape = trace_to_mlir(self.user_function, *self.c_sig)
    557 inject_functions(mlir_module, ctx)
    558 self._jaxpr = jaxpr

File ~/Code/catalyst/frontend/catalyst/jax_tracer.py:276, in trace_to_mlir(func, *args, **kwargs)
    273 mlir_fn_cache.clear()
    275 with EvaluationContext(EvaluationMode.CLASSICAL_COMPILATION):
--> 276     jaxpr, shape = jax.make_jaxpr(func, return_shape=True)(*args, **kwargs)
    278 return jaxpr_to_mlir(func.__name__, jaxpr, shape)

[... skipping hidden 6 frame]

File ~/Code/catalyst/frontend/catalyst/pennylane_extensions.py:149, in QFunc.__call__(self, *args, **kwargs)
    146 device = self.device
    148 with EvaluationContext(EvaluationMode.QUANTUM_COMPILATION):
--> 149     jaxpr, shape = trace_quantum_function(self.func, device, args, kwargs)
    151 retval_tree = tree_structure(shape)
    153 def _eval_jaxpr(*args):

File ~/Code/catalyst/frontend/catalyst/jax_tracer.py:535, in trace_quantum_function(f, device, args, kwargs)
    532 in_classical_tracers = _input_type_to_tracers(trace.new_arg, in_avals)
    533 with QueuingManager.stop_recording(), quantum_tape:
    534     # Quantum tape transformations happen at the end of tracing
--> 535     ans = wffa.call_wrapped(*in_classical_tracers)
    536     out_classical_tracers_or_measurements = [
    537         (trace.full_raise(t) if isinstance(t, DynamicJaxprTracer) else t) for t in ans
    538     ]
    540 # (2) - Quantum tracing
```

```

[... skipping hidden 1 frame]

File ~/tmp/autograph_generated_fileeeja8hs.py:9, in outer_factory.<locals>.inner_factory.<locals>.test_function
7 with ag_.FunctionScope('test_function', 'fscope', ag_.ConversionOptions(recursive=True, user_request
8 ag_.converted_call(subroutine, (dev.wires,), None, fscope)
----> 9 ag_.converted_call(ag_.converted_call(adjoint, (subroutine,), None, fscope), (dev.wires,), None,
10 return ag_.converted_call(qml.probs, (), None, fscope)

File ~/Code/catalyst/frontend/catalyst/ag_primitives.py:441, in converted_call(fn, args, kwargs, caller_fn_scope, options)
438 new_qnode = qml.QNode(qnode_call_wrapper, device=fn.device, diff_method=fn.diff_method)
439 return new_qnode()
--> 441 return tf_converted_call(fn, args, kwargs, caller_fn_scope, options)

File ~/Software/anaconda3/envs/catalyst/lib/python3.11/site-packages/tensorflow/python/autograph/impl/api.py:374
374 return _call_unconverted(f, args, kwargs, options)
376 if not options.user_requested and conversion.is_allowlisted(f):
--> 377 return _call_unconverted(f, args, kwargs, options)
379 # internal_convert_user_code is for example turned off when issuing a dynamic
380 # call conversion from generated code while in nonrecursive mode. In that
381 # case we evidently don't want to recurse, but we still have to convert
382 # things like builtins.
383 if not options.internal_convert_user_code:

File ~/Software/anaconda3/envs/catalyst/lib/python3.11/site-packages/tensorflow/python/autograph/impl/api.py:458
458 if kwargs is not None:
459 return f(*args, **kwargs)
--> 460 return f(*args)

File ~/Code/catalyst/frontend/catalyst/pennylane_extensions.py:1814, in adjoint.<locals>._callable(*args, **kwargs)
1813 def _callable(*args, **kwargs):
-> 1814 return _call_handler(*args, _callee=f, **kwargs)

File ~/Code/catalyst/frontend/catalyst/pennylane_extensions.py:1788, in adjoint.<locals>._call_handler(_callee, args, kwargs)
1786 with EvaluationContext.frame_tracing_context(ctx) as inner_trace:
1787 in_classical_tracers, _ = tree_flatten((args, kwargs))
-> 1788 wffa, in_avals, _ = deduce_avals(_callee, args, kwargs)
1789 arg_classical_tracers = _input_type_to_tracers(inner_trace.new_arg, in_avals)
1790 quantum_tape = QuantumTape()

File ~/Code/catalyst/frontend/catalyst/utils/jax_extras.py:266, in deduce_avals(f, args, kwargs)
264 flat_args, in_tree = tree_flatten((args, kwargs))
265 wf = wrap_init(f)
--> 266 in_avals, keep_inputs = list(map(shaped_abstractify, flat_args)), [True] * len(flat_args)
267 in_type = tuple(zip(in_avals, keep_inputs))
268 wff, out_tree_promise = flatten_fun(wf, in_tree)

[... skipping hidden 1 frame]

File ~/Software/anaconda3/envs/catalyst/lib/python3.11/site-packages/jax/_src/api_util.py:572, in _shaped_abstractify(x, dtype)
570 dtype = dtypes.canonicalize_dtype(x.dtype, allow_extended_dtype=True)
571 else:
--> 572 raise TypeError(
573     f"Cannot interpret value of type {type(x)} as an abstract array; it "
574     "does not have a dtype attribute")
575 return core.ShapedArray(np.shape(x), dtype, weak_type=weak_type,
576                          named_shape=named_shape)

TypeError: Cannot interpret value of type <class 'pennylane.wires.Wires'> as an abstract array; it does not have a dtype attribute

```

Additionally, if we pass the wires as a `jnp.array`, a separate error occurs in the adjoint call:

```

-----
TypeError                                Traceback (most recent call last)
Cell In[18], line 5
      2 for wire in wires:
      3     qml.PauliX(wire)
----> 5 @qjit(autograph=True)
      6 @qml.qnode(dev)
      7 def test_function():
      8     #subroutine(dev.wires)
      9     adjoint(subroutine)(jnp.array(dev.wires))
     10     return qml.probs()

File ~/Code/catalyst/frontend/catalyst/compilation_pipelines.py:948, in qjit.<locals>.wrap_fn(fn)
947 def wrap_fn(fn):
--> 948     return QJIT(
949         fn, CompileOptions(verbose, logfile, target, keep_intermediate, pipelines, autograph)
950     )

File ~/Code/catalyst/frontend/catalyst/compilation_pipelines.py:508, in QJIT.__init__(self, fn, compile_options)
506 if parameter_types is not None:

```

```

507     self.user_typed = True
--> 508     self.mlir_module = self.get_mlir(*parameter_types)
509     if self.compile_options.target == "binary":
510         self.compiled_function = self.compile()

File ~/Code/catalyst/frontend/catalyst/compilation_pipelines.py:555, in QJIT.get_mlir(self, *args)
550 self.c_sig = CompiledFunction.get_runtime_signature(*args)
552 with Patcher(
553     (qml.QNode, "__call__", QFunc.__call__),
554 ):
--> 555     mlir_module, ctx, jaxpr, self.shape = trace_to_mlir(self.user_function, *self.c_sig)
557 inject_functions(mlir_module, ctx)
558 self._jaxpr = jaxpr

File ~/Code/catalyst/frontend/catalyst/jax_tracer.py:276, in trace_to_mlir(func, *args, **kwargs)
273 mlir_fn_cache.clear()
275 with EvaluationContext(EvaluationMode.CLASSICAL_COMPILATION):
--> 276     jaxpr, shape = jax.make_jaxpr(func, return_shape=True)(*args, **kwargs)
278 return jaxpr_to_mlir(func.__name__, jaxpr, shape)

[... skipping hidden 6 frame]

File ~/Code/catalyst/frontend/catalyst/pennylane_extensions.py:149, in QFunc.__call__(self, *args, **kwargs)
146     device = self.device
148 with EvaluationContext(EvaluationMode.QUANTUM_COMPILATION):
--> 149     jaxpr, shape = trace_quantum_function(self.func, device, args, kwargs)
151 ret_val_tree = tree_structure(shape)
153 def _eval_jaxpr(*args):

File ~/Code/catalyst/frontend/catalyst/jax_tracer.py:535, in trace_quantum_function(f, device, args, kwargs)
532     in_classical_tracers = _input_type_to_tracers(trace.new_arg, in_aval)
533     with QueuingManager.stop_recording(), quantum_tape:
534         # Quantum tape transformations happen at the end of tracing
--> 535         ans = wffa.call_wrapped(*in_classical_tracers)
536         out_classical_tracers_or_measurements = [
537             (trace.full_raise(t) if isinstance(t, DynamicJaxprTracer) else t) for t in ans
538         ]
540 # (2) - Quantum tracing

[... skipping hidden 1 frame]

File /tmp/autograph_generated_filezwu16kl_.py:8, in outer_factory.<locals>.inner_factory.<locals>.test_function
6 def test_function_1():
7     with ag__.FunctionScope('test_function', 'fscope', ag__.ConversionOptions(recursive=True, user_rec
----> 8         ag__.converted_call(ag__.converted_call(adjoint, (subroutine,), None, fscope), (ag__.converted
9         return ag__.converted_call(qml.probs, (), None, fscope)

File ~/Code/catalyst/frontend/catalyst/ag_primitives.py:441, in converted_call(fn, args, kwargs, caller_fn_scope)
438     new_qnode = qml.QNode(qnode_call_wrapper, device=fn.device, diff_method=fn.diff_method)
439     return new_qnode()
--> 441 return tf_converted_call(fn, args, kwargs, caller_fn_scope, options)

File ~/Software/anaconda3/envs/catalyst/lib/python3.11/site-packages/tensorflow/python/autograph/impl/api.py:3
374     return _call_unconverted(f, args, kwargs, options)
376 if not options.user_requested and conversion.is_allowlisted(f):
--> 377     return _call_unconverted(f, args, kwargs, options)
379 # internal_convert_user_code is for example turned off when issuing a dynamic
380 # call conversion from generated code while in nonrecursive mode. In that
381 # case we evidently don't want to recurse, but we still have to convert
382 # things like builtins.
383 if not options.internal_convert_user_code:

File ~/Software/anaconda3/envs/catalyst/lib/python3.11/site-packages/tensorflow/python/autograph/impl/api.py:4
458 if kwargs is not None:
459     return f(*args, **kwargs)
--> 460 return f(*args)

File ~/Software/anaconda3/envs/catalyst/lib/python3.11/site-packages/jax/_src/numpy/lax_numpy.py:2051, in arra
2044 out: ArrayLike
2046 if all(not isinstance(leaf, Array) for leaf in leaves):
2047     # TODO(jakevdp): falling back to numpy here fails to overflow for lists
2048     # containing large integers; see discussion in
2049     # https://github.com/google/jax/pull/6047. More correct would be to call
2050     # coerce_to_array on each leaf, but this may have performance implications.
--> 2051     out = np.array(object, dtype=dtype, ndmin=ndmin, copy=False) # type: ignore[arg-type]
2052 elif isinstance(object, Array):
2053     assert object.aval is not None

TypeError: Wires.__array__() takes 1 positional argument but 2 were given

```

Additional information

Any additional information, configuration or data that might be necessary to reproduce the issue.



 **dime10** added **bug** on Nov 8, 2023



sergei-mironov on Jan 3, 2024

Contributor ...

FYI: the following workaround should work

```
from functools import partial

def subroutine(*, wires):
    for wire in wires:
        qml.PauliX(wire)

dev = qml.device("lightning.qubit", wires=3)

@qjit.autograph=True
@qml.qnode(dev)
def test_function():
    adjoint(partial(subroutine, wires=dev.wires))()
    return qml.probs()

test_function()
```



 **sergei-mironov** mentioned this on Jan 5, 2024

[Support PyTree API for Wires objects PennyLaneAI/pennylane#4998](#)



 **sergei-mironov** added a commit that references this issue on Jan 8, 2024

Support PyTree API for Wires objects ([#4998](#)) ...

Verified 5d4d8bd



 **sergei-mironov** mentioned this on Jan 8, 2024

[\[Tests\] Check the PyTree-compatibility of Wires #432](#)



 **dime10** closed this as **completed** in [#432](#) on Jan 16, 2024



 **dime10** added a commit that references this issue on Jan 16, 2024

[Tests] Check the PyTree-compatibility of Wires ([#432](#)) ...

Verified d21f46e

Support PyTree API for Wires objects #4998

<> Code

Merged

sergei-mironov merged 8 commits into `v0.34.0-rc0` from `wires-pytree` on Jan 8, 2024

Conversation 10

Commits 8

Checks 33

Files changed 3

+34 -3



sergei-mironov commented on Jan 5, 2024 · edited

Contributor

Context:

In order to pass objects to Catalyst (`@eqjit`)-compiled functions, objects should be Jax-compatible or support the PyTree serialization API. The issue [PennyLaneAI/catalyst#334](#) highlighted the fact that Wires class does not match these requirements, as well as the use-case where it could actually be useful.

Description of the Change:

In this PR we add the PyTree serialization API to the `Wires` class.

Benefits:

Wires objects could be passed as a positional argument to a Catalyst-compiled function.

Possible Drawbacks:

- Not known

Related GitHub Issues:

- [\[BUG\] Error when taking adjoint of subroutines with wires arguments catalyst#334](#)

[\[sc-48585\]](#)



Reviewers

[dime10](#)



[albi3ro](#)



Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

3 participants



github-actions bot commented on Jan 5, 2024

Contributor

Hello. You may have forgotten to update the changelog!

Please edit [doc/releases/changelog-dev.md](#) with:

- A one-to-two sentence description of the change. You may include a small working example for new features.
- A link back to this PR.
- Your name (or GitHub username) in the contributors section.



codecov bot commented on Jan 5, 2024 · edited

...

Codecov Report

All modified and coverable lines are covered by tests

No coverage uploaded for pull request base (`v0.34.0-rc0@9d9df4`). [Click here to learn what that means.](#)

► Additional details and impacted files

[View full report in Codecov by Sentry.](#)

Have feedback on the report? [Share it here.](#)

